



MongoDB Dasar

Eko Kurniawan Khannedy

Eko Kurniawan Khannedy

- Technical architect at one of the biggest ecommerce company in Indonesia
- 12+ years experiences
- www.programmerzamannow.com
- youtube.com/c/ProgrammerZamanNow





Eko Kurniawan Khannedy

- Telegram : [@khannedy](https://t.me/khannedy)
- LinkedIn : <https://www.linkedin.com/company/programmer-zaman-now/>
- Facebook : fb.com/ProgrammerZamanNow
- Instagram : instagram.com/programmerzamannow
- Youtube : youtube.com/c/ProgrammerZamanNow
- Telegram Channel : t.me/ProgrammerZamanNow
- Tiktok : <https://tiktok.com/@programmerzamannow>
- Email : echo.khannedy@gmail.com



Sebelum Belajar

- Mengerti cara menggunakan perintah berbasis terminal
- Mengerti salah satu database relational, misal MySQL atau PostgreSQL
- Mengerti bahasa pemrograman JavaScript sangat direkomendasikan

Pengenalan MongoDB



Pengenalan MongoDB

- MongoDB merupakan free dan opensource database management system
- MongoDB merupakan database management system berbasis document
- Dikembangkan oleh perusahaan bernama 10gen tahun 2007
- Dirilis ke public tahun 2009
- Saat ini perusahaan 10gen sudah berganti nama menjadi MongoDB Inc
- MongoDB hampir mendukung semua bahasa pemrograman sebagai client nya
- MongoDB tidak menggunakan SQL, namun menggunakan JavaScript sebagai bahasa utama untuk manipulasi document
- <https://github.com/mongodb/mongo>

db-engines.com/en/ranking/document+store

☐ include secondary database models

47 systems in ranking, May 2020

Rank			DBMS	Database Model	Score		
May 2020	Apr 2020	May 2019			May 2020	Apr 2020	May 2019
1.	1.	1.	MongoDB	Document, Multi-model	438.99	+0.57	+30.92
2.	2.	2.	Amazon DynamoDB	Multi-model	64.72	+0.45	+8.78
3.	3.	4.	Microsoft Azure Cosmos DB	Multi-model	30.68	-1.37	+3.08
4.	4.	3.	Couchbase	Document, Multi-model	28.58	-1.83	-6.09
5.	5.	5.	CouchDB	Document	16.92	-0.85	-2.19
6.	6.	7.	Firebase Realtime Database	Document	13.12	+0.47	+1.84
7.	7.	6.	MarkLogic	Multi-model	10.96	-0.30	-3.09
8.	8.	8.	Realm	Document	8.38	-0.16	+0.74
9.	9.	10.	Google Cloud Firestore	Document	6.34	-0.27	+1.35
10.	10.	11.	ArangoDB	Multi-model	4.68	-0.20	-0.11



Apa itu Document Oriented Database

- Document oriented database merupakan sistem database yang digunakan untuk memanipulasi data dalam bentuk document (semi structured data)
- Biasanya document disimpan dalam bentuk JSON atau XML
- Document oriented database biasanya bertolak belakang dengan relational database.
- Relational database biasanya menyimpan data dalam bentuk table, dan menyimpan relasinya di table lain.
- Document oriented database biasanya menyimpan data dalam bentuk JSON atau XML, dan menyimpan relasinya sebagai embedded object di dalam document yang sama.



Istilah Relational DB vs Document DB

Relational DB	Document DB (MongoDB)
Database	Database
Table	Collection
Column	Field
Row, Record	Document (JSON, XML, dan lain-lain)
Join Table	Embedded Document, Reference
SQL	JavaScript (MongoDB)

Menginstall MongoDB



Menginstall MongoDB

- <https://www.mongodb.com/try/download/community>
- Download MongoDB dalam bentuk archive file
- Lalu extract archive file tersebut di komputer kita
- Di dalam archive file tersebut akan terdapat file binary dengan nama file bin/mongod, yaitu mongo daemon, untuk digunakan menjalankan aplikasi MongoDB Server



Menjalankan MongoDB

- Buat folder untuk menyimpan data MongoDB
- Gunakan perintah `bin/mongod --dbpath=lokasi/folder/data`
- Untuk menghentikannya, cukup gunakan perintah `Ctrl + C`

MongoDB Compass



MongoDB Client

- Tidak seperti aplikasi database lainnya (MySQL / PostgreSQL), MongoDB hanya berjalan sebagai aplikasi Server saja
- Jika kita ingin terkoneksi ke MongoDB, kita harus menggunakan aplikasi client nya
- Ada banyak pilihan yang bisa kita gunakan untuk menggunakan aplikasi client, ada yang berbasis terminal, atau GUI (Graphical User Interface)
- Salah satu yang disediakan oleh MongoDB adalah MongoDB Compass



MongoDB Compass

- MongoDB Compass adalah aplikasi berbasis GUI yang bisa kita gunakan secara gratis sebagai aplikasi client untuk MongoDB
- Selain itu, salah satu hal yang bagus di MongoDB Compass, dia mendukung perintah Terminal, sehingga kita bisa mengetikkan perintah untuk dijalankan di MongoDB
- Silahkan download dan install aplikasi MongoDB Compass sebelum melanjutkan materi ini
- <https://www.mongodb.com/products/compass>

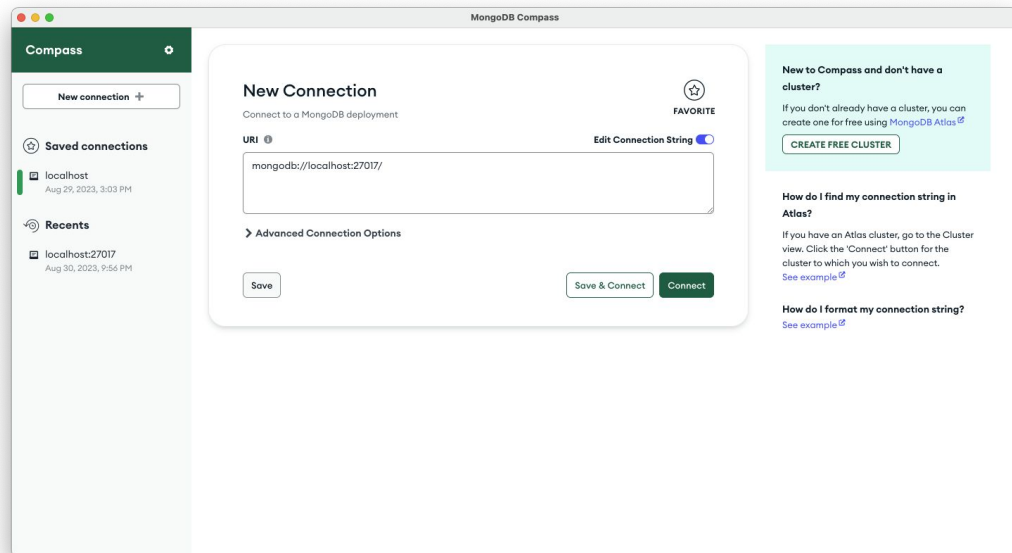
MongoDB Client



Mongo Client

- Setelah kita menginstall aplikasi client seperti MongoDB Compass, kita bisa mulai melakukan koneksi dari aplikasi client ke MongoDB Server
- Secara default, MongoDB akan berjalan menggunakan port 27017

MongoDB Compass



MongoDB Shell



MongoDB Shell

- Pada kasus kita menggunakan sistem operasi yang berbasis terminal, seperti Linux Server, maka kita tidak bisa menggunakan aplikasi berbasis GUI seperti MongoDB Compass
- MongoDB juga menyediakan aplikasi client berbasis terminal, bernama MongoDB Shell
- Kita bisa download dan menggunakannya secara gratis
- <https://www.mongodb.com/products/shell>



Menggunakan MongoDB Shell

- Untuk menggunakan MongoDB Shell, kita cukup extract archive file yang sudah kita download, lalu didalamnya terdapat file bin/mongosh
- Kita bisa gunakan perintah berikut untuk terkoneksi ke MongoDB Server
- `bin/mongosh mongodb://host:port/namadatabase`

Database



Database

- Database adalah tempat menyimpan collection
- Semua collection harus disimpan di database
- Biasanya database digunakan untuk memisahkan data secara logical per aplikasi, artinya biasanya satu aplikasi akan memiliki satu database
- Jarang sekali kita akan menggunakan satu database untuk beberapa aplikasi



Membuat Database

- Kita tidak perlu secara eksplisit membuat database
- MongoDB akan secara otomatis membuatkan database sesuai dengan nama database yang kita pilih
- Untuk memilih nama database, kita bisa menggunakan perintah “use” diikuti nama database



JS database.js

```
// memilih database

use belajar // memilih database belajar

use hello // memilih database hello

show databases // menampilkan semua databases
```



Database Methods

Database Methods	Keterangan
db.dropDatabase()	Menghapus database
db.getName()	Mengambil nama database
db.hostInfo()	Mengambil informasi host tempat mongodb
db.version()	Mengambil versi database
db.stats()	Mengambil statistik penggunaan database

Collection



Collection

- Collection adalah tempat menyimpan document
- Maximum per document yang bisa disimpan adalah 16MB
- Maximum level nested document yang bisa disimpan adalah 100 level



Database Methods untuk Collection

Database Methods untuk Collection	Keterangan
<code>db.getCollectionNames()</code>	Mengambil semua nama collection
<code>db.createCollection(name)</code>	Membuat collection baru
<code>db.getCollection(name)</code>	Mendapatkan object collection
<code>db.<name></code>	Sama dengan <code>db.getCollection(<name>)</code>
<code>db.getCollectionInfos()</code>	Mendapat informasi semua collection



Collection Methods

Database Methods untuk Collection	Keterangan
db.<collection>.find()	Mengambil semua document
db.<collection>.count()	Mengambil jumlah document
db.<collection>.drop()	Menghapus collection
db.<collection>.totalSize()	Mengambil total ukuran collection
db.<collection>.stats()	Mengambil informasi statistik collection



JS collection.js

```
// Buatlah beberapa collection

db.createCollection("customers") // membuat collection customers

db.createCollection("products") // membuat collection products

db.createCollection("orders") // membuat collection orders
```

Data Model



Kenapa Perlu Mengerti Data Modeling

- Pindah dari relational database ke document database bukanlah hal yang sesederhana hanya dengan memindahkan semua table ke collection
- Penggunaan document database tidak akan mendatangkan manfaat besar jika kita tidak mengerti cara memodelkan data untuk kebutuhan aplikasi kita
- Saat memodelkan data menggunakan relational database, biasanya kita mengacu ke database normalization
- Saat memodelkan data menggunakan document database, kita harus mengacu ke penggunaan aplikasi dalam melakukan query, update dan memproses data



Schema yang Flexible

- Tidak seperti di relational database, di MongoDB kita bisa memasukkan data ke collection secara langsung tanpa mendefinisikan schema collection nya.
- Schema untuk collection di MongoDB sangat flexible, tiap document bisa berbeda. Tidak seperti table di relational database yang harus sama tiap record.
- Namun pada prakteknya, sangat direkomendasikan menggunakan jenis data yang sama untuk tiap collection, walaupun bisa berbeda-beda di collection yang sama



Primary Key

- Saat membuat dokumen di MongoDB, kita wajib menambahkan primary key
- Tidak seperti relational database yang bebas membuat column untuk primary key, di MongoDB, primary key wajib menggunakan field _id
- Selain itu primary key tidak bisa lebih dari 1 field, hanya bisa field _id, jadi jika kita ingin membuat composite primary key, maka kita hanya bisa melakukan dengan menggunakan 1 field _id

Struktur Document Embedded

```
data.json
{
  "_id" : "123123123",
  "name" : "Eko Kurniawan",
  "address" : {
    "street" : "Jalan belum ada",
    "city" : "Jakarta",
    "country" : "Indonesia",
  },
  "contact" : {
    "email" : "eko@pzn.com",
    "phone" : "6211111111"
  },
  "hobbies" : [
    "Coding", "Gaming", "Reading"
  ]
}
```

Embedded Sub Document

Embedded Sub Document

Struktur Document Reference

```
user.json
{
  "_id" : "eko",
  "name" : "Eko Kurniawan"
}
```

```
address.json
{
  "_id" : "21312321",
  "user_id" : "eko",
  "street" : "Jalan belum ada",
  "city" : "Jakarta",
  "country" : "Indonesia",
}
```

```
contact.json
{
  "_id" : "2343434",
  "user_id" : "eko",
  "email" : "eko@pzn.com",
  "phone" : "6211111111"
}
```





Embedded vs Reference

Gunakan Embedded jika :

- Antar document saling ketergantungan
- Tidak bisa langsung melakukan perubahan ke embedded document
- Embedded document selalu dibutuhkan ketika mengambil data document

Gunakan Reference jika :

- Antar document bisa berdiri sendiri dan tidak terlalu ketergantungan satu sama lain
- Bisa melakukan manipulasi data langsung terhadap reference document
- Reference document tidak selalu dibutuhkan saat mengambil document

BSON



BSON

- BSON singkatan dari Binary JSON, yaitu binary-encoded serialization dokumen seperti JSON
- Sama seperti JSON, di BSON juga bisa kita bisa menggunakan embedded object, array dan lain-lain
- <http://bsonspec.org/>
- <https://docs.mongodb.com/manual/reference/bson-types/>



Type Data di BSON (1)

Type Data	Alias
Double	double
String	string
Object	object
Array	arrat
Binary Data	binData
ObjectId	objectId



Type Data di BSON (2)

Type Data	Alias
Boolean	bool
Date	date
Null	null
Regular Expression	regex
JavaScript	javascript
JavaScript with Scope	javascriptWithScope



Tipe Data di BSON (3)

Tipe Data	Alias
32 Bit Integer	int
Timestamp	timestamp
64 Bit Integer	long
Decimal 128	decimal
Min Key	minKey
Max key	maxKey



ObjectId

- ObjectId adalah random data yang unik, cepat untuk digenerate dan terurut.
- Nilai ObjectId memiliki ukuran panjang 12 byte, konsisten terdiri dari informasi 4 byte timestamp, 5 byte random value, dan 3 byte incrementing counter
- ObjectId digunakan sebagai default `_id` (primary key) di document jika kita tidak secara eksplisit menyebutkan `_id` document nya



Date dan ISODate

- BSON Date adalah 64 bit integer yang merepresentasikan angka milisecond sejak Unix epoch (1 Januari 1970).
- ISODate merupakan representasi waktu yang digunakan oleh MongoDB
- Date ini kompatibel dengan Date di JavaScript
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

Insert Document



Insert Document

- Untuk menyimpan data ke MongoDB, kita perlu membuat document dalam bentuk JSON
- Field `_id` tidak wajib dimasukkan, jika kita tidak memasukkan field `_id`, maka secara otomatis MongoDB akan membuat `_id` baru secara random dengan tipe data `ObjectId`
- Atau kita juga bisa secara eksplisit membuat `ObjectId` baru dengan menggunakan perintah “new `ObjectId()`”



Insert Document Function

Function	Keterangan
<code>db.<collection>.insertOne(document)</code>	Menambah dokumen ke collection
<code>db.<collection>.insertMany(array<document>)</code>	Menambah semua dokumen di array ke collection

JS customers.js

```
// insert customers document
db.customers.insertOne({
  _id: "khannedy",
  name: "Eko Kurniawan Khannedy",
});
```

JS products.js

```
// insert product documents
db.products.insertMany([
  {
    _id: 1,
    name: "Indomie Ayam Bawang",
    price: new NumberLong("2000")
  },
  {
    _id: 2,
    name: "Mie Sedap Soto",
    price: new NumberLong("2000")
  }
]);
```

JS orders.js

```
// insert orders document
db.orders.insertOne({
  _id: new ObjectId(),
  total: new NumberLong("8000"),
  items: [
    {
      product_id: 1,
      price: new NumberLong("2000"),
      quantity: new NumberInt("2")
    },
    {
      product_id: 2,
      price: new NumberLong("2000"),
      quantity: new NumberInt("2")
    }
  ]
});
```

Query Document



Query Document

- Sama seperti di relational database, di MongoDB pun kita bisa melakukan query atau pencarian document yang sudah kita simpan di collection



Query Document Function

Function	Keterangan
<code>db.<collection>.find(query)</code>	Mencari document dengan query



JS query.js

```
// select * from customers where _id = "khannedy"
db.customers.find({_id:"khannedy"});

// select * from customers where name = "Eko Kurniawan Khannedy"
db.customers.find({name:"Eko Kurniawan Khannedy"});

// select * from products where price = 2000
db.products.find({price: 2000});

// select * from orders where items.product_id = 1
db.orders.find({"items.product_id": 1});
```

Comparison Query Operator



Comparison Operator (1)

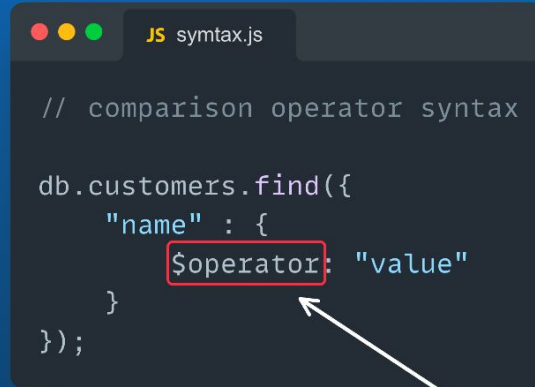
Operator	Keterangan
\$eq	Membandingkan value dengan value lain
\$gt	Membandingkan value lebih besar dari value lain
\$gte	Membandingkan value lebih besar atau sama dengan value lain
\$lt	Membandingkan value lebih kecil dari value lain
\$lte	Membandingkan value lebih kecil atau sama dengan value lain



Comparison Operator (2)

Operator	Keterangan
\$in	Membandingkan value dengan value yang ada di array
\$nin	Membandingkan value tidak ada dalam value yang ada di array
\$ne	Membandingkan value tidak sama dengan value lain

Syntax Comparison Operator



```
// comparison operator syntax

db.customers.find({
  "name" : {
    $operator: "value"
  }
});
```

Ubah menjadi comparison operator

JS products.js

```
// insert product documents
db.products.insertMany([
  {
    _id: 3,
    name: "Pop Mie Rasa Bakso",
    price: new NumberLong("2500"),
    category: "food"
  },
  {
    _id: 4,
    name: "Samsung Galaxy S9+",
    price: new NumberLong("10000000"),
    category: "handphone"
  },
  {
    _id: 5,
    name: "Acer Precator XXI",
    price: new NumberLong("25000000"),
    category: "laptop"
  }
]);
```

JS customer-query.js

```
// select * from customers where _id = "khannedy"
db.customers.find({
  _id: {
    $eq: "khannedy"
  }
});
```

JS product-query.js

```
// select * from products where price > 1000
db.products.find({
  price: {
    $gt: 1000
  }
});

// select * from products where category in ('handphone', 'laptop')
// and price > 5000000
db.products.find({
  category: {
    $in: ["handphone", "laptop"]
  },
  price: {
    $gt: 5000000
  }
});
```

Logical Query Operator



Logical Operator

Operator	Keterangan
\$and	Menggabungkan query dengan operasi AND, mengembalikan document jika semua kondisi benar
\$or	Menggabungkan query dengan operasi OR, mengembalikan document jika salah satu kondisi benar
\$nor	Menggabungkan query dengan operasi NOR, mengembalikan document yang gagal di semua kondisi
\$not	Membalikkan kondisi, mengembalikan document yang tidak sesuai kondisi

Logical Operator Syntax

```
JS logical-operator.js

// logical operator for $and, $or and $nor
db.customers.find({
  $operator: [
    {
      // query
    },
    {
      // query
    }
  ]
});

// logical operator for $not
db.customers.find({
  field: {
    $not: {
      // query
    }
  }
});
```

Gunakan \$and, \$or atau \$nor



JS logical-operator.js

```
// select * from products where cateogry in ('handphone', 'laptop') and price > 200000000
db.products.find({
  $and: [
    {category: {$in: ["handphone", "laptop"]}},
    {price: {$gt: 200000000}}
  ]
});

// select * from productw where category not in ('handphone', 'laptop')
db.products.find({
  category: {
    $not: {$in: ["handphone", "laptop"]}
  }
});

// select * from products where price between 100000000 and 200000000 and category ≠ 'food'
db.products.find({
  price: {
    $gte: 100000000,
    $lte: 200000000
  },
  category: {
    $ne: "food"
  }
});
```

Element Query Operator



Element Operator

Operator	Keterangan
\$exists	Mencocokkan document yang memiliki field tersebut
\$type	Mencocokkan document yang memiliki type field tersebut

Element Operator Syntax



A code editor window titled "JS element-operator.js" displays a MongoDB query. The query is: `db.customers.find({ field: { $operator: value } });`. The text `$operator` is highlighted with a red rectangular box. A white arrow points from a text box on the right towards this box.

```
db.customers.find({  
  field: {  
    $operator: value  
  }  
});
```

Ubah menjadi element operator



JS element-operator.js

```
// select * from products where category is null
db.products.find({
  category: {
    $exists: false
  }
});

// select * from productw where type(category) = 'string'
db.products.find({
  category: {
    $type: "string"
  }
});

// select * from products where type(price) in ('int', 'long')
db.products.find({
  price: {
    $type: ["int", "long"]
  }
});
```

Evaluation Query Operator



Evaluation Operator

Operator	Keterangan
\$expr	Menggunakan aggregation operation
\$jsonSchema	Validasi document sesuai dengan JSON schema : https://json-schema.org/
\$mod	Melakukan operasi modulo
\$regex	Mengambil document sesuai dengan regular expression (PCRE)
\$text	Melakukan pencarian menggunakan text
\$where	Mengambil document dengan JavaScript Function

\$expr Operator

```
JS expr-query.js

// insert customers documents
db.customers.insertOne({
  _id: "joko",
  name: "joko",
});

// select * from customers where _id = name
db.customers.find({
  $expr : {
    $eq: ["$_id", "$name"]
  }
});
```

\$jsonSchema Operator

JS jsonschema-query.js

```
// select * from products where name is not null and category is not null
db.products.find({
  $jsonSchema: {
    required: [ "name", "category"]
  }
});

// select * from products where name is not null and type(name) = 'string' and type(price) = 'number'
db.products.find({
  $jsonSchema: {
    required: [ "name"],
    properties: {
      name: {
        type: "string"
      },
      price: {
        type: "number"
      }
    }
  }
});
```

\$mod Operator

```
JS mod-query.js

// select * from products where price % 5 = 0
db.products.find({
  price: {
    $mod: [5, 0]
  }
});

// select * from products where price % 1000000 = 0
db.products.find({
  price: {
    $mod: [1000000, 0]
  }
});
```

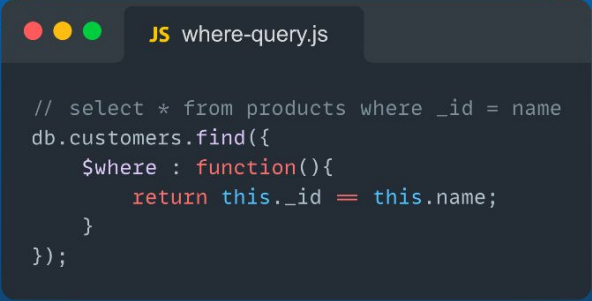
\$regex Operator

```
JS regex-query.js

// select * from products where name like "%mie%"
db.products.find({
  name: {
    $regex: /mie/,
    $options: "i"
  }
});

// select * from products where name like "Mie%"
db.products.find({
  name: {
    $regex: /^Mie/
  }
});
```


\$where Operator



```
// select * from products where _id = name
db.customers.find({
  $where : function(){
    return this._id = this.name;
  }
});
```



\$text Operator

- Untuk menggunakan \$text operator, kita harus mengerti tentang Text Index terlebih dahulu
- Oleh karena ini, \$text operator akan kita bahas di materi Text Index

Array Query Operator



Array Operator

Operator	Keterangan
\$all	Mencocokkan array yang mengandung elemen-elemen tertentu
\$elemMatch	Mengambil document jika tiap element di array memenuhi kondisi tertentu
\$size	Mengambil document jika ukuran array sesuai

JS products.js

```
// insert products with tags
db.products.insertMany([
  {
    _id: 6,
    name: "Logitech Wireless Mouse",
    price: new NumberLong("175000"),
    category: "laptop",
    tags: ["logitech", "mouse", "accessories"]
  },
  {
    _id: 7,
    name: "Cooler Pad Gaming",
    price: new NumberLong("200000"),
    category: "laptop",
    tags: ["cooler", "laptop", "accessories", "fan"]
  },
  {
    _id: 8,
    name: "Samsung Curved Monitor ",
    price: new NumberLong("1750000"),
    category: "computer",
    tags: ["samsung", "monitor", "computer"]
  }
]);
```

JS array-query.js

```
// select * from products where (tags = "samsung" and tags = "monitor")
db.products.find({
  tags: {
    $all: ["samsung", "monitor"]
  }
});

// select * from products where tags in ("samsung", "logitech")
db.products.find({
  tags: {
    $elemMatch: {
      $in: ["samsung", "logitech"]
    }
  }
});

// select * from products where count(tags) = 3
db.products.find({
  tags: {
    $size: 3
  }
});
```

Projection Operator



Projection

- Pada function find, terdapat parameter kedua setelah query, yaitu projection
- Projection adalah memilih field mana yang ingin kita ambil atau hide
- `db.<collection>.find(query, projection)`

Projection

JS projection.js

```
// select _id, name, category from products
db.products.find({}, {
  name : 1,
  category: 1
});

// select _id, name, category, price from products
db.products.find({}, {
  tags: 0
});
```




Projection Operator

Operator	Keterangan
\$	Limit array hanya mengembalikan data pertama yang match dengan array operator
\$elemMatch	Limit array hanya mengembalikan data pertama yang match dengan kondisi query
\$meta	Mengembalikan informasi metadata yang didapat dari setiap matching document
\$slice	Mengontrol jumlah data yang ditampilkan pada array

\$elemMatch Operator

```
JS elem-match.js

// select _id, name, tags[first] from products
db.products.find({}, {
  name: 1,
  tags: {
    $elemMatch: {
      $in: ["samsung", "logitech"]
    }
  }
})
```

\$ Operator

JS \$-operator.js

```
// select _id, name, tags[first] from products where tags is not null
db.products.find({
  tags: {
    $exists: true
  }
}, {
  name: 1,
  "tags.$": 1
})
```

\$slice Operator

```
JS slice-operator.js

// select _id, name, category, price, tags[0-1] from products where tags is not null
db.products.find({
  tags: {
    $exists: true
  }
}, {
  name: 1,
  tags: {
    $slice: 2
  }
})
```



\$meta Operator

- Untuk menggunakan \$meta operator, kita harus membuat Text Index terlebih dahulu
- Materi ini akan kita bahas di materi Text Index

Query Modifier



Query Modifier

- Query Modifier adalah memodifikasi hasil query yang telah kita lakukan
- Contoh yang sering kita lakukan seperti, mengubah query menjadi jumlah data, membatasi jumlah data dengan paging, dan lain-lain
- Untuk memodifikasi hasil query, kita bisa menambahkan function query modifier setelah menggunakan function find



Query Modifier Function

Operator	Keterangan
count()	Mengambil jumlah data hasil query
limit(size)	Membatasi jumlah data yang didapat dari query
skip(size)	Menghiraukan data pertama hasil query sejumlah yang ditentukan
sort(query)	Mengurutkan hasil data query

Query Modifier



JS query-modifier.js

```
// select count(*) from products
db.products.find({}).count()

// select * from products limit 4
db.products.find({}).limit(4)

// select * from products offset 2
db.products.find({}).skip(2)

// select * from products limit 4 offset 2
db.products.find({}).limit(4).skip(2)

// select * from products order by category asc, name desc
db.products.find({}).sort({
  category: 1,
  name: -1
})
```

Update Document



Update Document

- Sama seperti database lainnya, di MongoDB juga kita bisa mengubah document yang sudah kita insert ke collection
- Namun berbeda dengan perintah SQL, di MongoDB, untuk mengubah document, kita diberikan beberapa function



Update Document Function

Operator	Keterangan
updateOne()	Mengubah satu document
updateMany()	Mengubah banyak document sekaligus
replaceOne()	Mengubah total satu document dengan document baru

Update Syntax

JS update-syntax.js

```
// update one syntax
db.customers.updateOne(
  {}, // filter
  {}, // update
  {} // options
);

// update many syntax
db.customers.updateMany(
  {}, // filter
  {}, // update
  {} // options
);

// replace one syntax
db.customers.replaceOne(
  {}, // filter
  {}, // update
  {} // options
);
```

updateOne



JS update-one.js

```
// update products set category = "food" where _id = 1
db.products.updateOne({
  _id: 1
},{
  $set: {
    category: "food"
  }
});

// update products set category = "food" where _id = 2
db.products.updateOne({
  _id: 2
},{
  $set: {
    category: "food"
  }
});
```

updateMany

```
JS update-many.js

// update products set tags = ["food"] where category = "food" and tags is null
db.products.updateMany({
  $and: [
    {
      category :{
        $eq: "food"
      }
    },
    {
      tags: {
        $exists: false
      }
    }
  ]
}, {
  $set: {
    tags: ["food"]
  }
})
```

replaceOne

```
JS replace-one.js

// insert wrong document
db.products.insertOne(
  {
    _id: 9,
    name: "Ups Salah",
    wrong: "Salah Lagi"
  }
);

// replace document with id 9
db.products.replaceOne({
  _id: 9
}, {
  name: "Adidas Sepatu lari Pria",
  price: new NumberLong("1100000"),
  category: "shoes",
  tags: [
    "adidas", "shoes", "running"
  ]
});
```

Field Update Operator



Field Update Operator

- Sebelumnya kita sudah tau kalo untuk update document di MongoDB kita bisa menggunakan operator \$set
- Namun sebenarnya masih banyak operator yang bisa kita gunakan



Update Document Function

Operator	Keterangan
\$set	Mengubah nilai field di document
\$unset	Menghapus field di document
\$rename	Mengubah nama field di document
\$inc	Menaikan nilai number di field sesuai dengan jumlah tertentu
\$currentDate	Mengubah field menjadi waktu saat ini

\$set

```
JS $set.js

// update products set stock = 0
db.products.updateMany({}, {
  $set: {
    stock: 0
  }
});
```

\$inc



```
// update products set stock = stock + 10
db.products.updateMany({}, {
  $inc: {
    stock: 10
  }
});
```

\$rename



JS rename.js

```
// alter table customers change name full_name
db.customers.updateMany({}, {
  $rename: {
    name: "full_name"
  }
});
```

\$unset

```
JS unset.js

// update customers set wrong = 'Ups'
db.customers.updateMany({}, {
  $set: {
    wrong: "ups"
  }
});

// alter table customers drop dolumn wrong
db.customers.updateMany({}, {
  $unset: {
    wrong: ""
  }
});
```

\$currentDate



JS current-date.js

```
// update products set lastModifiedDate = current_date()
db.products.updateMany({}, {
  $currentDate: {
    lastModifiedDate: {
      $type: "date"
    }
  }
});
```

Array Update Operator



Array Update Operator

- Secara default, saat kita mengubah field dengan tipe array, maka seluruh array akan diubah
- Kadang kita ingin menambah, atau hanya mengubah data array tanpa harus mengubah seluruh field array
- Hal ini bisa dilakukan di MongoDB



Array Update Operator (1)

Operator	Keterangan
\$	Mengupdate data array pertama sesuai kondisi query
\$[]	Mengupdate semua data array sesuai kondisi query
\$[<identifier>]	Mengupdate semua data array yang sesuai kondisi arrayFilters
<index>	Mengupdate data array sesuai dengan nomor index



Array Update Operator (2)

Operator	Keterangan
\$addToSet	Menambahkan value ke array, dihiraukan jika sudah ada
\$pop	Menghapus element pertama (-1) atau terakhir (1) array
\$pull	Menghapus semua element di array yang sesuai kondisi
\$push	Menambahkan element ke array
\$pullAll	Menghapus semua element di array

Array \$ Operator

JS array-\$-operator.js

```
// update products set ratings = [90, 80, 70]
db.products.updateMany({}, {
  $set: {
    ratings: [90, 80, 70]
  }
});

// update first element of array, query must include array fields
db.products.updateMany({
  ratings: 90
}, {
  $set: {
    "ratings.$": 100
  }
});
```

Array \$[] Operator

```
JS array-$[]-operator.js

// update all element of array to 100
db.products.updateMany({}, {
  $set: {
    "ratings.$[]": 100
  }
})
```

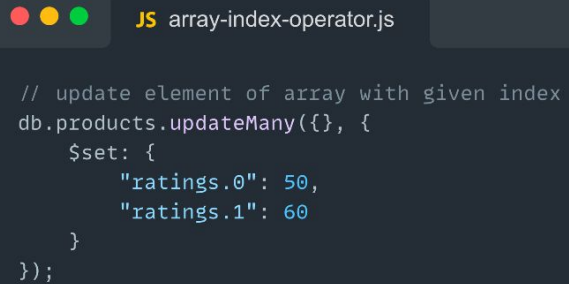
Array \$[<identifier>] Operator

```
JS array-$[<identifier>]-operator.js

// update products set ratings = [90, 80, 70]
db.products.updateMany({}, {
  $set: {
    ratings: [90, 80, 70]
  }
});

// update element of array based on arrayFilters
db.products.updateMany({}, {
  $set: {
    "ratings.$[element]" : 100
  }
},{
  arrayFilters: [
    {
      element: {
        $gte: 80
      }
    }
  ]
})
```

Array <index> Operator



A code editor window with a dark theme. The title bar shows three colored window control buttons (red, yellow, green) and a tab labeled "JS array-index-operator.js". The code is written in a light gray font on a dark background.

```
// update element of array with given index
db.products.updateMany({}, {
  $set: {
    "ratings.0": 50,
    "ratings.1": 60
  }
});
```


Array \$addToSet Operator

```
JS add-to-set.js

// add "popular" to array if not exists
db.products.updateOne({
  _id: 1
}, {
  $addToSet: {
    tags: "popular"
  }
});
```

Array \$pop Operator

```
JS pop.js

// remove first element of array
db.products.updateOne({
  _id: 1
}, {
  $pop: {
    ratings: -1
  }
});

// remove last element of array
db.products.updateOne({
  _id: 2
}, {
  $pop: {
    ratings: 1
  }
});
```

Array \$pull, \$push dan \$pushAll Operator

JS pull.js

```
// update products set rating = [90, 80, 70]
db.products.updateMany({}, {
  $set: {
    ratings: [90, 80, 70]
  }
});

// remove all element where ratings ≥ 80
db.products.updateMany({}, {
  $pull: {
    ratings: {
      $gte: 80
    }
  }
})
```

JS push.js

```
// add 100 to ratings
db.products.updateMany({}, {
  $push: {
    ratings: 100
  }
})
```

JS pushAll.js

```
// remove element 100
db.products.updateMany({}, {
  $pullAll: {
    ratings: [100]
  }
})
```



Array Update Operator Modifier

Operator	Keterangan
\$each	Digunakan di \$addToSet dan \$push, untuk menambahkan multiple element
\$position	Digunakan di \$push untuk mengubah posisi menambahkan data
\$slice	Digunakan di \$push untuk menentukan jumlah maksimal data array
\$sort	Digunakan untuk mengurutkan array setelah operasi \$push

Array Update Operator Modifier

JS each.js

```
// add 100, 200, 300 to ratings
db.products.updateMany({}, {
  $push: {
    ratings: {
      $each: [100, 200, 300]
    }
  }
})

// add trending, popular to tags
db.products.updateMany({}, {
  $addToSet: {
    tags: {
      $each: ["trending", "popular"]
    }
  }
})
```

JS position.js

```
// add hot in position 1
db.products.updateMany({}, {
  $push: {
    tags: {
      $each: ["hot"],
      $position: 1
    }
  }
})
```

JS slice.js

```
// add all element, but limit with slice from behind
db.products.updateMany({}, {
  $push: {
    ratings: {
      $each: [100, 200, 300, 400, 500],
      $slice: -3
    }
  }
})
```

JS sort.js

```
// add all element, and sort desc
db.products.updateMany({}, {
  $push: {
    ratings: {
      $each: [100, 200, 300, 400, 500],
      $sort: -1
    }
  }
})
```

Delete Document



Delete Document

- MongoDB memiliki function yang bisa kita gunakan untuk menghapus document di collection secara permanen
- Document yang sudah kita hapus, tidak akan bisa dikembalikan lagi



Delete Document Function

Function	Keterangan
<code>db.<collection>.deleteOne(query)</code>	Menghapus satu document yang sesuai dengan kondisi query
<code>db.<collection>.deleteMany(query)</code>	Menghapus banyak document yang sesuai dengan kondisi query

Delete Operation

JS delete-one.js

```
// Insert spammer document
db.customers.insertOne({
  _id: "spammer",
  full_name: "Spammer"
});

// Delete document by _id
db.customers.deleteOne({
  _id: "spammer"
});
```

JS delete-many.js

```
// Insert many spammer documents
db.customers.insertMany([
  {
    _id: "spammer1",
    full_name: "Spammer1"
  },
  {
    _id: "spammer2",
    full_name: "Spammer2"
  },
  {
    _id: "spammer3",
    full_name: "Spammer3"
  }
]);

// Delete many documents
db.customers.deleteMany({
  _id: {
    $regex: "spammer"
  }
});
```

Bulk Write Operations



Bulk Write Operation

- Komunikasi antara aplikasi dengan database biasanya dilakukan secara request-response
- Artinya tiap perintah yang ingin kita kirimkan dari aplikasi ke database, akan di-response secara langsung oleh database
- Proses tersebut akan sangat lambat, jika kita menghadapi kasus harus memanipulasi data besar secara langsung. Misal upload data dari file dengan jumlah jutaan ke dalam database.
- MongoDB mendukung Bulk Write Operation, yaitu operasi bulk yang dalam satu request, kita bisa mengirim banyak perintah
- Fitur ini cocok pada kasus jika kita ingin melakukan operasi bulk atau batch secara banyak sekaligus



Bulk Write Function

Function	Keterangan
<code>db.<collection>.insertMany()</code>	Insert document secara banyak sekaligus
<code>db.<collection>.updateMany()</code>	Update document secara banyak sekaligus
<code>db.<collection>.deleteMany()</code>	Delete document secara banyak sekaligus
<code>db.<collection>.bulkWrite()</code>	Melakukan operasi write (insert, update, delete) banyak secara sekaligus



Supported Bulk Write Operation

- insertOne
- updateOne
- updateMany
- replaceOne
- deleteOne
- deleteMany

Bulk Write

JS bulk-write.js

```
db.customers.bulkWrite([
  {
    insertOne: {
      document: {
        _id: "eko",
        full_name: "Eko"
      }
    }
  },
  {
    insertOne: {
      document: {
        _id: "kurniawan",
        full_name: "Kurniawan"
      }
    }
  },
  {
    updateMany: {
      filter: {
        _id: {
          $in: ["eko", "kurniawan", "khannedy"]
        }
      },
      update: {
        $set: {
          full_name: "Eko Kurniawan Khannedy"
        }
      }
    }
  }
])
```

Indexes



Indexes

- Index adalah fitur di MongoDB untuk mengefisienkan proses query. Tanpa Index, MongoDB harus melakukan proses query dengan cara collection scan (mencari keseluruhan data dari awal sampai akhir)
- Jika terdapat Index pada collection MongoDB, MongoDB bisa menggunakan Index untuk mendapatkan data, tanpa harus melakukan pencarian keseluruhan document
- Index menggunakan struktur data khusus ketika menyimpan data, defaultnya menggunakan BTREE (balanced tree)
- Index menyimpan spesifik field, lalu mengurutkan data field tersebut. Hal ini tidak hanya mempermudah ketika proses pencarian, namun mempermudah ketika kita butuh melakukan pencarian dalam bentuk range document (seperti paging).
- Secara sederhana, Index di MongoDB, cara kerjanya sama seperti Index di Relational DB



Create Index Function

Function	Keterangan
<code>db.<collection>.createIndex()</code>	Membuat index di collection
<code>db.<collection>.getIndexes()</code>	Melihat semua index di collection
<code>db.<collection>.dropIndex()</code>	Menghapus index di collection



Single Field Index

- Secara default, field `_id` di MongoDB sudah di index secara otomatis, jadi kita tidak perlu membuat index lagi secara manual untuk field `_id`
- MongoDB mendukung penuh pembuatan index di semua field yang ada di document. Dengan begitu, ini bisa mempercepat proses query di MongoDB untuk query terhadap field tersebut
- Saat membuat index, kita bisa menggunakan nilai `1` untuk ascending, dan `-1` untuk descending

Single Index

JS single-index.js

```
// Create index at category in products collection
db.products.createIndex({
  category: 1
});

// Get all indexes in products collection
db.products.getIndexes();

// Find products by category (will use index)
db.products.find({
  category: "food"
});
```

JS debug-single-index.js

```
// debugging with index
db.products.find({
  category: "food"
}).explain();
db.products.find({}).sort({
  category: 1
}).explain();

// debugging without index
db.products.find({
  "tag" : "laptop"
}).explain()
```



Compound Indexes

- Jika kita butuh melakukan query terhadap lebih dari satu field, kita juga bisa membuat index terhadap lebih dari satu field, atau disebut Compound Index
- MongoDB membatasi pembuatan maksimal field yang bisa dibuat di compound index adalah 32 field

Compound Index

JS compound-index.js

```
// Create index at price and tags in products collection
db.products.createIndex({
  stock: 1,
  tags: 1
});

// Find products by stock and tags (will use index)
db.products.find({
  stock: 10,
  tags: "popular"
});
```

JS debug-compound-index.js

```
// debug with index
db.products.find({
  stock: 10
}).explain();
db.products.find({
  stock: 10,
  tags: "popular"
}).explain();

// debug without index
db.products.find({
  tags: "popular"
}).explain();
```



Index Strategy

- Buat index untuk mendukung performa query
 - Gunakan single index, jika kita hanya melakukan query terhadap satu field saja
 - Gunakan compound index, jika kita sering melakukan query ke field pertama, atau kombinasi field pertama dan kedua, atau pertama dan kedua dan seterusnya
- Buat index untuk mengurutkan hasil query
- Sering-seringlah menggunakan function explain() untuk mengecek apakah query kita sudah di optimize dengan index atau belum
- Jika kita buat compound index (a, b, c), artinya bisa mencari menggunakan query a, a b, dan a b c

Text Indexes



Text Indexes

- MongoDB menyediakan text index untuk mendukung pencarian text di tipe data string.
- Text index tidak hanya bisa digunakan pada field dengan tipe data string, namun juga pada array berisi tipe data string
- Text index lebih cepat dibanding menggunakan \$regex, karena \$regex akan melakukan pencarian scanning sehingga semakin besar data semakin banyak
- Walaupun fitur Text Index bagus, tapi fitur nya tidak secanggih Search Engine, oleh karena itu sangat direkomendasikan menggunakan database khusus Search Engine jika kita butuh membuat fitur pencarian layaknya Search Engine, contohnya database Elasticsearch

Text Index

JS text-index.js

```
// create index text
db.products.createIndex({
  name: "text",
  category: "text",
  tags: "text"
}, {
  weights: {
    name: 10,
    category: 5,
    tags: 1
  }
});

// get index
db.products.getIndexes();
```

JS search.js

```
// search products with text "mie"
db.products.find({
  $text: {
    $search: "mie"
  }
});

// search products with text "mie" OR "laptop"
db.products.find({
  $text: {
    $search: "mie laptop"
  }
});

// search products with text "mie sedap"
db.products.find({
  $text: {
    $search: '"mie sedap"'
  }
});

// search products with text "mie" and NOT "sedap"
db.products.find({
  $text: {
    $search: "mie -sedap"
  }
});
```

JS debug.js

```
// Debugging query optimization
db.products.find({
  $text: {
    $search: "mie -sedap"
  }
}).explain();
```



\$meta Operator

- Dalam operasi \$text, \$meta bisa digunakan untuk mendapatkan text score hasil pencarian
- Ini bermanfaat untuk membantu proses debugging

\$meta Operator

```
JS meta.js

db.products.find({
  $text: {
    $search: "mie"
  }
}, {
  searchScore: {
    $meta: "textScore"
  }
});
```

Wildcard Indexes



Wildcard Indexes

- MongoDB mendukung wildcard index, dimana ini digunakan untuk membuat index terhadap field yang belum diketahui atau field yang sering berubah-ubah
- Misal contoh kita punya sebuah embedded document dengan tipe field customFields, dimana isinya bisa bebas sesuai dengan data yang dimasukkan.
- Agar bisa mendukung proses query yang cepat pada field tersebut, kita bisa menggunakan wildcard index

Wildcard Index

JS wildcard-index.js

```
// membuat wildcard index
db.customers.createIndex({
  "customFields.$**" : 1
});

// mengambil index
db.customers.getIndexes()
```

JS insert.js

```
// Insert many customers
db.customers.insertMany([
  {
    _id: "budi",
    full_name: "Budi",
    customFields: {
      hobby: "Gaming",
      university: "Universitas Belum Ada"
    }
  },
  {
    _id: "rully",
    full_name: "rully",
    customFields: {
      ipk: 3.2,
      university: "Universitas Belum Ada"
    }
  },
  {
    _id: "rudi",
    full_name: "Rudi",
    customFields: {
      motherName: "Tini",
      passion: "Entrepreneur"
    }
  }
])
```

JS debug.js

```
// Debug wildcard index
db.customers.find({
  "customFields.passion": "Entrepreneur"
}).explain();

db.customers.find({
  "customFields.ipk": 3.2
}).explain();

db.customers.find({
  "customFields.hobby": "Gaming"
}).explain();
```

Index Properties



Index Properties

- MongoDB mendukung properties di index
- Istilah properties di Index mungkin agak sedikit membingungkan, sederhananya adalah menambahkan kemampuan tertentu terhadap index yang kita buat



TTL Index

- TTL singkatan dari Time To Live, yaitu waktu untuk hidup
- TTL Index digunakan sebagai waktu hidup document di collection, artinya data akan hilang dalam kurun waktu tertentu secara otomatis
- TTL Index hanya dapat digunakan di field dengan tipe data Date
- Background proses di MongoDB akan secara regular melakukan penghapusan data secara otomatis
- Sayangnya, proses background tersebut berjalan setiap 60 detik sekali

TTL

JS ttl.js

```
// Create session collection
db.createCollection("sessions");

// create TTL index
db.sessions.createIndex({
  createdAt: 1
}, {
  expireAfterSeconds: 10
});

// Will expire after 10 seconds, but background job run every 60 seconds
db.sessions.insertOne({
  _id: 1,
  session: "Session 1",
  createdAt: new Date()
});
```



Unique Index

- Unique Index memastikan bahwa field-field di index tersebut tidak menyimpan data duplicate.
- Contohnya, di MongoDB, field `_id` secara otomatis merupakan unique index, sehingga kita tidak bisa membuat document dengan field `_id` yang sama

Unique

```
JS unique.js

// Create unique index in email
db.customers.createIndex({
  email: 1
}, {
  unique: true,
  sparse: true
});
```

Index hanya diterapkan
pada dokumen yang memiliki
field yang di index

```
JS update-email.js

// update customers set email = ? where _id = ?
db.customers.updateOne({
  _id: "eko"
}, {
  $set : {
    email : "eko@example.com"
  }
});

// error duplicate email
db.customers.updateOne({
  _id: "joko"
}, {
  $set : {
    email : "eko@example.com"
  }
});
```



Case Insensitive

- Secara default, saat kita membuat index, maka data akan disimpan sebagai case sensitive
- Namun, kadang ada kasus kita ingin membuat data yang di index menjadi case insensitive
- Kita bisa mengubah nilai collation strength saat membuat index

Case Insensitive

```
JS case-insensitive.js

// Create unique index in full_name
db.customers.createIndex({
  full_name: 1
}, {
  collation: {
    locale: "en",
    strength: 2
  }
});
```

```
JS debug.js

// not using index
db.customers.find({
  full_name: "Eko Kurniawan Khannedy"
});

// using index
db.customers.find({
  full_name: "EKO KURNIAWAN KHANNEDY"
}).collation({
  locale: "en",
  strength: 2
});
```



Partial

- Secara default, saat kita membuat jenis query apapun yang menggunakan query ke index, secara otomatis mongodb akan menggunakan index
- Namun kita bisa menambahkan kondisi tertentu agar index hanya digunakan ketika kita menggunakan kriteria tertentu ketika melakukan query
- Fitur ini bernama partial filter

Partial

JS partial-index.js

```
// create partial index
db.products.createIndex({
  price: 1
}, {
  partialFilterExpression: {
    stock: {
      $gt: 0
    }
  }
});
```

JS debug.js

```
// debug query with partial index
db.products.find({
  price: {
    $gt: 1000
  },
  stock : {
    $gt: 0
  }
}).explain()
```

Security



Security

- Secara default, jika kita menjalankan MongoDB, mode yang dijalankan tidaklah aman
- Tidak ada Authentication dan tidak ada Authorization
- Agar aman, kita harus mengaktifkan fitur access control di MongoDB



User Admin

- User admin harus ada terlebih dahulu sebelum kita mengaktifkan access control
- User admin adalah user yang memiliki role `userAdminAnyDatabase` dan `readWriteAnyDatabase`
- Setelah membuat user admin, kita bisa menjalankan ulang MongoDB dengan perintah `--auth`

Admin User

JS admin.js

```
// switch to database admin
use admin;

// create admin user
db.createUser(
{
  user: 'mongo',
  pwd: 'mongo',
  roles: [
    'userAdminAnyDatabase',
    'readWriteAnyDatabase'
  ]
}
);
```



Menjalankan Ulang Mongodb

- Gunakan perintah seperti berikut untuk mengaktifkan fitur Security
- `bin/mongod --auth --dbpath=lokasi/data/`



MongoDB Client

- Sekarang, untuk terkoneksi ke MongoDB Server menggunakan MongoDB Client, kita wajib menggunakan username dan password yang sudah kita buat sebelumnya
- Gunakan koneksi URL seperti berikut :
bin/mongosh "mongodb://username:password@host:port/database?authSource=admin"

—

User



Authentication

- Authentication adalah proses memverifikasi identitas pengguna ketika mengakses MongoDB
- Saat menggunakan authentication, maka client wajib menggunakan username dan password untuk terkoneksi ke MongoDB server



User

- Di MongoDB, kita bisa menambahkan user, dan juga menambahkan role ke user tersebut
- Saat kita membuat user, kita harus menentukan database sebagai authentication database
- Namun bukan berarti user hanya bisa mengakses database itu saja, tapi user juga bisa mengakses database lain jika mau
- Nama user harus unik per database, namun jika database nya berbeda, kita bisa membuat user dengan nama yang sama



User Function

Function	Keterangan
db.createUser()	Membuat user
db.getUsers()	Mendapatkan semua user
db.dropUser()	Menghapus user
db.updateUser()	Mengupdate user
db.changeUserPassword()	Mengubah user password



Authorization

- Authorization adalah proses yang dilakukan setelah proses Authentication sukses
- Authorization dilakukan untuk melakukan pengecekan apakah user memiliki hak akses untuk melakukan sebuah action
- Hak akses di MongoDB disimpan dalam bentuk role



Database Roles

Role	Keterangan
read	Bisa membaca data di semua collection yang bukan sistem collection
readWrite	Bisa membaca dan mengubah data di semua collection yang bukan sistem collection
dbAdmin	Bisa melakukan kemampuan administrasi database
userAdmin	Mampu membuat user dan role
dbOwner	Kombinasi readWrite, dbAdmin dan userAdmin



All Database Roles

Role	Keterangan
readAnyDatabase	Seperti read role, tapi untuk semua database
readWriteAnyDatabase	Seperti readWrite role, tapi untuk semua database
userAdminAnyDatabase	Seperti userAdmin, tapi untuk semua database
dbAdminAnyDatabase	Seperti dbAdmin, tapi untuk semua database



Backup & Restore Roles

Role	Keterangan
backup	Kemampuan untuk melakukan backup database
restore	Kemampuan untuk melakukan restore database



Superuser Roles

Role	Keterangan
root	Bisa melakukan apapun

User Management

JS user.js

```
use admin;

// Create user with access read only
db.createUser(
{
  user: "contoh",
  pwd: "contoh",
  roles: [
    { role: "read", db: "test" }
  ]
}
);

// Create user with access read write
db.createUser(
{
  user: "contoh2",
  pwd: "contoh2",
  roles: [
    { role: "readWrite", db: "test" }
  ]
}
)
```

JS change-user.js

```
// Get all users
db.getUsers()

// Change password for user contoh
db.changeUserPassword("contoh", "rahasia")

// Drop user contoh
db.dropUser("contoh")

// Update user
db.updateUser("contoh2",
{
  roles: [
    { role: "readWrite", db: "test" },
    { role: "readWrite", db: "belajar" }
  ]
}
)
```

Role



Custom Role

- Sebenarnya menggunakan Built In Role sudah cukup ketika membuat user, namun pada kasus tertentu jika kita ingin membuat Role sendiri, itu juga bisa kita lakukan di MongoDB
- Built In Role kebanyakan membatasi hak akses di level database
- Kadang kita ingin membatasi di level collection
- Untuk melakukan itu, kita bisa menggunakan privileges



Role Function

Role	Keterangan
db.createRole()	Membuat role baru
db.getRoles()	Mendapatkan role
db.deleteRole()	Menghapus role
db.updateRole()	Mengubah role

Role Management

JS role.js

```
// create role with custom privileges
db.createRole({
  role: "session_management",
  privileges: [
    {
      resource: {
        db: "belajar",
        collection: "sessions"
      },
      actions: [ "insert" ]
    }
  ],
  roles: [
    {
      role: "read",
      db: "belajar"
    }
  ]
});

// show all roles with privileges
db.getRoles({showPrivileges:true});
```

JS user.js

```
// create user with custom role
db.createUser({
  user: "eko",
  pwd: "eko",
  roles: [ "session_management" ]
});
```

JS test.js

```
// login using user eko
// find sessions (success)
db.sessions.find();

// insert session (success)
db.sessions.insertOne({
  _id: "test",
  name: "test"
});

// delete session (error)
db.sessions.deleteOne({
  _id: "test"
});

// update session (error)
db.sessions.updateOne({
  _id: "test"
}, {
  $set: {
    name: "ubah"
  }
});
```



Backup



MongoDB Tools

- MongoDB tidak memiliki bawaan aplikasi untuk melakukan backup dan restore
- Tapi untungnya, MongoDB juga menyediakan aplikasi lain yang bisa digunakan untuk mempermudah kita melakukan backup dan restore, namanya adalah MongoDB Tools
- Ini adalah aplikasi berbasis terminal yang bisa kita download secara gratis
- <https://www.mongodb.com/try/download/database-tools>



Aplikasi MongoDB Tools

- Di dalam MongoDB Tools terdapat beberapa aplikasi yang bisa kita gunakan untuk melakukan backup
- mongodump adalah aplikasi yang digunakan untuk melakukan backup dalam format binary, cocok untuk database yang berisi data binary atau ukuran yang sangat besar
- mongoexport adalah aplikasi yang digunakan untuk melakukan backup dalam format JSON / CSV, cocok untuk jenis database yang hanya berisi teks dan berukuran tidak terlalu besar, dan hanya bisa per collection



Backup Menggunakan mongodump

bin/mongodump

--uri="mongodb://mongo:mongo@localhost:27017/belajar?authSource=admin"

--out=backup-dump



Backup Menggunakan mongoexport

```
bin/mongoexport  
--uri="mongodb://mongo:mongo@localhost:27017/belajar?authSource=admin"  
--collection=customers  
--out=customers.json
```

Restore



Aplikasi MongoDB Tools

- Untuk melakukan restore, kita bisa menggunakan aplikasi tergantung saat kita melakukan backup
- mongorestore digunakan untuk melakukan restore hasil backup mongodump
- mongoimport digunakan untuk melakukan restore hasil backup mongoexport



Restore Menggunakan mongorestore

bin/mongorestore

--uri="mongodb://mongo:mongo@localhost:27017/belajar_restore?authSource=admin"

--dir=backup-dump/belajar



Restore Menggunakan mongoimport

bin/mongoimport

--uri="mongodb://mongo:mongo@localhost:27017/belajar_import?authSource=admin"

--collection=customers --file=customers.json

Materi Selanjutnya



Materi Selanjutnya

- MongoDB Schema Validation
- MongoDB Geospatial
- MongoDB Aggregation
- MongoDB Time Series
- MongoDB Replication
- MongoDB Sharding